

## xQueueCreate

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength,
                           UBaseType_t uxItemSize );
```

- Creates a new queue and returns a handle by which the queue can be referenced
- Parameters:
  - uxQueueLength
    - The maximum number of items the queue can hold at any one time.
  - uxItemSize
    - The size, in bytes, required to hold each item in the queue.
- Returns:
  - If the queue is created successfully then a handle to the created queue is returned.
  - If the memory required to create the queue could not be allocated then NULL is returned.

## xQueueSendToBack

```
BaseType_t xQueueSendToBack(
    QueueHandle_t xQueue,
    const void * pvItemToQueue,
    TickType_t xTicksToWait
);
```

- Post an item to the back of a queue. The item is queued by copy, not by reference.
- This function must not be called from an interrupt service routine.
- xQueueSendToBackFromISR() for an alternative which may be used in an ISR.
- Parameters:
  - xQueue
    - The handle to the queue on which the item is to be posted.
  - pvItemToQueue
    - A pointer to the item that is to be placed on the queue.
  - xTicksToWait
    - The maximum amount of time the task should block waiting for space to become available on the queue
- Returns:
  - pdPASS if the item was successfully posted, otherwise errQUEUE\_FULL.
- xQueueSendToBackFromISR

```

BaseType_t xQueueSendToBackFromISR
(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken
);

```

- Post an item to the back of a queue.
- It is safe to use this function from within an interrupt service routine.
- Parameters:
  - xQueue
    - The handle to the queue on which the item is to be posted.
  - pvItemToQueue
    - A pointer to the item that is to be placed on the queue.
  - pxHigherPriorityTaskWoken
    - will set \*pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task.
- Returns:
  - pdPASS if sending to the queue was successful, otherwise errQUEUE\_FULL.

## xQueueSendToFront

```

BaseType_t xQueueSendToFront( QueueHandle_t xQueue,
    const void * pvItemToQueue,
    TickType_t xTicksToWait );

```

- Post an item to the front of a queue. The item is queued by copy, not by reference.
- This function must not be called from an interrupt service routine.
- xQueueSendToFrontFromISR() for an alternative which may be used in an ISR.
- Parameters:
  - xQueue
    - The handle to the queue on which the item is to be posted.
  - pvItemToQueue
    - A pointer to the item that is to be placed on the queue.
  - xTicksToWait
    - The maximum amount of time the task should block waiting for space to become available on the queue
- Returns:
  - pdPASS if the item was successfully posted, otherwise errQUEUE\_FULL.

## xQueueSendToFrontFromISR

```

BaseType_t xQueueSendToFrontFromISR
(

```

```
QueueHandle_t xQueue,
const void *pvItemToQueue,
BaseType_t *pxHigherPriorityTaskWoken
);
```

- Post an item to the front of a queue.
- It is safe to use this function from within an interrupt service routine.
- Parameters:
  - xQueue
    - The handle to the queue on which the item is to be posted.
  - pvItemToQueue
    - A pointer to the item that is to be placed on the queue.
  - pxHigherPriorityTaskWoken
    - will set \*pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task.
- Returns:
  - pdPASS if sending to the queue was successful, otherwise errQUEUE\_FULL.

## xQueueReceive

```
BaseType_t xQueueReceive(
    QueueHandle_t xQueue,
    void *pvBuffer,
    TickType_t xTicksToWait
);
```

- Receive an item from a queue.
- The item is received by copy so a buffer of adequate size must be provided.
- Parameters:
  - xQueue
    - The handle to the queue from which the item is to be received.
  - pvBuffer
    - Pointer to the buffer into which the received item will be copied.
  - xTicksToWait
    - The maximum amount of time the task should block waiting for an item to receive
- Returns:
  - pdPASS if an item was successfully received from the queue, errQUEUE\_EMPTY otherwise.

## xQueueReceiveFromISR

```
BaseType_t xQueueReceiveFromISR
(
    QueueHandle_t xQueue,
    void *pvBuffer,
    BaseType_t *pxHigherPriorityTaskWoken
);
```

- Receive an item from a queue. It is safe to use this function from within an interrupt service routine.
- Parameters:
  - xQueue
    - The handle to the queue from which the item is to be received.
  - pvBuffer
    - Pointer to the buffer into which the received item will be copied.
  - pxHigherPriorityTaskWoken
    - A task may be blocked waiting for space to become available on the queue.
- Returns:
  - pdPASS if an item was successfully received from the queue, pdFAIL otherwise.

## xQueuePeek

```
BaseType_t xQueuePeek(
QueueHandle_t xQueue,
void *pvBuffer,
TickType_t xTicksToWait
);
```

- Receive an item from a queue without removing the item from the queue.
- Parameters:
  - xQueue
    - The handle to the queue from which the item is to be received.
  - pvBuffer
    - Pointer to the buffer into which the received item will be copied.
  - xTicksToWait
    - The maximum amount of time the task should block waiting for an item to receive
- Returns:
  - pdPASS if an item was successfully received (peeked) from the queue, otherwise errQUEUE\_EMPTY.

## taskENTER\_CRITICAL(), taskEXIT\_CRITICAL()

```
void taskENTER_CRITICAL( void );
void taskEXIT_CRITICAL( void );
```

- Critical sections are entered by calling taskENTER\_CRITICAL(), and subsequently exited by calling taskEXIT\_CRITICAL().
- The taskENTER\_CRITICAL() and taskEXIT\_CRITICAL() macros provide a basic critical section implementation that works by simply disabling interrupts, either globally, or up to a specific interrupt priority level.

## taskYIELD

```
taskYIELD()
```

- used to request a context switch to another task.
- if there are no other tasks at a higher or equal priority to the task that calls taskYIELD() then the RTOS scheduler will simply select the task that called taskYIELD() to run again.

SUNBEAM